面试精选百题-AI应用与智能体开发 v1.0

一、大模型与 AI 应用基础

Q1：什么是大语言模型？它和传统机器学习模型最大的区别是什么？

大语言模型本质上是参数规模很大、基于海量语料预训练出来的生成式模型，核心能力是根据上下文预测下一个最合理的 token。它和传统机器学习模型最大的区别在于，传统模型通常针对单一任务训练，比如分类、回归、推荐，而大语言模型先做通用预训练，再通过提示词、检索、工具调用或微调去适配很多任务，所以泛化能力更强，迁移成本更低。

Q2：为什么 Transformer 会成为大模型的主流架构？

核心原因是 Transformer 用自注意力机制替代了 RNN 的时序递归，训练时可以高度并行，更适合在大规模 GPU 集群上扩展。它还能直接建模长距离依赖，不像 RNN 那样容易出现长期依赖衰减问题。再加上结构统一、可扩展到文本、图像、语音、多模态，所以它成了大模型时代最主流的底层架构。

Q3：Self-Attention 相比 RNN 和 CNN 的优势是什么？

Self-Attention 的优势主要有三个。第一，它可以直接看到整个序列，建模远距离依赖更自然；第二，它可以并行计算，不像 RNN 必须一步一步处理；第三，它会根据当前输入动态分配不同位置的重要性，表达能力更强。它的主要代价是序列长度很长时计算和显存开销会明显上升，因为标准注意力复杂度接近 $O(n^2)$。

Q4：预训练、SFT、RLHF、DPO 分别是什么？

预训练是在海量通用语料上学习语言规律和世界知识，让模型先具备基础理解与生成能力。SFT 就是监督微调，用高质量问答或指令数据把模型往特定风格和任务方向拉齐。RLHF 是先训练奖励模型，再通过强化学习让模型输出更符合人类偏好。DPO 可以理解为不显式做强化学习、直接用偏好对比数据优化策略，工程实现更简单，最近在很多场景里比 RLHF 更常见。

Q5：temperature、top_p、max_tokens 这些参数分别控制什么？

temperature 控制随机性，越低越稳定，越高越发散；top_p 是核采样，表示只从累计概率达到某个阈值的一小部分候选 token 里采样；max_tokens 控制本次最多生成多少内容。面试里可以直接说，temperature 影响创造性和确定性，top_p 影响采样范围，max_tokens 影响输出长度和成本，这三个参数通常需要结合具体业务一起调。

Q6：大模型为什么会出现幻觉？怎么缓解？

幻觉的根本原因是模型本质上是在做概率生成，不是真正去数据库里查事实，所以当训练知识不完整、上下文不充分、提示词不清晰时，它就可能生成看起来很像真的错误答案。缓解方式通常有几类：第一，用 RAG 提供外部知识；第二，限制输出格式并要求引用来源；第三，降低温度，减少发散；第四，对高风险场景增加规则校验、人审和工具验证。我的理解是，幻觉无法完全消灭，只能通过系统设计把风险降下来。

Q7：什么是上下文窗口？什么是 KV Cache？

上下文窗口指模型单次推理时能看到的最大 token 范围，窗口越大，模型一次能参考的信息就越多，但成本和时延通常也会更高。KV Cache 是生成阶段的一种加速机制，模型在前面 token 上算出来的 Key 和 Value 会缓存起来，后续生成新 token 时不用重复计算整段上下文，所以能显著降低解码时延。简单说，上下文窗口解决“能看多长”，KV Cache 解决“生成时怎么算得更快”。

Q8：Embedding 和 Rerank 的区别是什么？

Embedding 主要解决召回问题，它把文本映射成向量，通过向量相似度快速找出一批“可能相关”的候选内容。Rerank 主要解决精排问题，它会把用户问题和候选文档一起再判断一遍，给出更精细的相关性排序。实际系统里通常是 Embedding 先做粗召回，保证速度，再用 Rerank 做精排，保证准确率。

Q9：量化、蒸馏、LoRA 分别适合什么场景？

量化是把模型权重从高精度改成低精度，主要目的是降低显存占用和推理成本，适合部署优化。蒸馏是让小模型学习大模型的能力，目标是做更轻量的替代方案，适合边缘端或高并发场景。LoRA 是一种参数高效微调方法，不改动全量参数，只训练少量增量参数，适合在有限算力下做领域定制。三者的关注点分别是部署压缩、模型缩小和低成本微调。

Q10：做知识型问答时，什么时候选微调，什么时候选 RAG？

如果问题主要是知识更新快、需要可追溯、需要引用企业内部文档，我会优先选 RAG，因为更新知识不需要重新训练模型，成本更低，也更容易做权限控制。如果问题主要是输出风格、领域术语、固定流程、工具选择习惯需要长期稳定下来，我会考虑微调。很多真实项目里不是二选一，而是先用微调把模型行为调顺，再用 RAG 解决知识实时性问题。

Q11：Tokenizer 为什么重要？

因为模型真正处理的不是字符也不是单词，而是 token。Tokenizer 会直接影响上下文长度、计费成本、分块策略以及中英文混合场景下的表现。同一段文本用不同 tokenizer 切出来的 token 数可能差很多，所以在做成本评估、长文本切分、截断策略时，必须先理解 tokenizer 的行为。

Q12：一个典型 AI 应用系统的整体架构一般是什么样的？

通常会分成几层：最上层是前端或业务入口；中间是应用编排层，负责提示词、会话状态、工具调用、权限控制和结果后处理；下面是知识与工具层，包括向量库、数据库、搜索引擎、业务 API；再下面是模型层，可以接公有云模型也可以接私有模型；旁边还要有日志、监控、缓存、鉴权、评测和成本统计。面试里我一般会强调，AI 应用不是只接一个模型 API，而是一个完整的工程系统。

二、Python 与 AI 开发工程能力

Q13：为什么 Python 在 AI 开发里这么常见？

因为 Python 生态非常成熟，PyTorch、Transformers、FastAPI、LangChain、数据处理和科学计算工具链都很完善，很多新模型和新论文也是先给 Python 版本。它的语法也足够简单，适合快速验证想法，所以在模型训练、推理服务、RAG 和智能体原型阶段，Python 的开发效率通常很高。

Q14：Python 做 AI 应用时常见的技术栈有哪些？

如果是模型和数据层，常见的是 PyTorch、Transformers、NumPy、Pandas。做 Web 服务常见的是 FastAPI、Uvicorn、Pydantic。做异步任务和队列常见的是 Celery、Redis、RabbitMQ。做 RAG 可能会配合 FAISS、Milvus、Qdrant、pgvector。做评测和实验管理时，也会用到 Weights & Biases、MLflow 或一些自定义埋点系统。

Q15：Python 的 GIL 对 AI 服务有什么影响？

GIL 的意思是同一个 Python 进程里，同一时刻只能有一个线程执行 Python 字节码，所以对 CPU 密集型任务，多线程并不能真正并行，收益有限。但对 I/O 密集型任务，比如调用外部模型 API、访问数据库、等待网络响应，多线程和异步仍然有价值。实际项目里，如果是 CPU 密集型预处理，我更倾向于多进程、C 扩展或者把重计算下沉到模型服务侧。

Q16：asyncio 在 AI 应用里适合解决什么问题？

它更适合处理高并发 I/O 场景，比如同时调用多个外部接口、并发召回多个知识源、做流式输出、管理大量网络连接。它不适合拿来硬做 CPU 重任务，因为那类问题瓶颈不在 I/O。我的经验是，智能体系统里经常会出现很多“等待外部工具返回”的阶段，这种时候 asyncio 会比纯同步代码更高效。

Q17：为什么很多 Python AI 服务喜欢用 FastAPI？

因为 FastAPI 天然支持类型标注、异步接口和自动生成 OpenAPI 文档，写接口快，和 Pydantic 配合后请求校验、响应约束都比较顺手。对于 AI 服务这种迭代快、接口多、经常需要定义结构化入参出参的场景，它的开发体验通常比传统框架更好。

Q18：在 Python 项目里，多线程、多进程、协程该怎么选？

如果主要是网络 I/O，比如调模型 API、查数据库、调搜索接口，我会优先考虑协程；如果需要利用多核做 CPU 密集型计算，我会选多进程；如果只是兼容一些阻塞库或者做轻量并发，我会考虑线程。面试里最好不要绝对化地说哪种方式更好，而是强调要看瓶颈到底在 CPU、I/O 还是工程复杂度。

Q19：Python AI 服务调用外部模型接口时，超时、重试、降级一般怎么设计？

我通常会先分清楚哪些错误可重试，哪些错误不可重试，比如网络抖动、临时 5xx 可以重试，参数错误就不该重试。然后会设置合理的连接超时和读取超时，避免请求长期挂死；重试要加退避和上限，避免雪崩；再往上要有降级策略，比如切备用模型、切简化回答模式，或者直接返回兜底提示。核心思想不是“尽量重试”，而是“让系统在失败时仍然可控”。

Q20：Pydantic 在 AI 应用里有什么价值？

它最大的价值是把原本很容易失控的 JSON 输入输出结构化、类型化。比如工具调用参数、模型结构化输出、知识检索返回格式，都可以先定义 Schema，再统一校验，这样可以减少脏数据流入下游系统。对智能体项目来说，Pydantic 不只是参数校验工具，本质上也是让系统边界更清晰的一层约束。

Q21：什么是批处理推理？什么时候值得做 Batch？

批处理推理就是把多个请求合并成一批一起送进模型，目的是提高吞吐和硬件利用率。它通常适合离线任务、高并发相似请求或者对实时性没那么敏感的场景。如果是严格低延迟的在线对话，Batch 做得太大反而会拉高单请求时延，所以要在吞吐和响应时间之间找平衡。

Q22：Python AI 服务里常见的内存问题有哪些？

比较常见的是大对象长期引用不释放、缓存无上限增长、异步任务堆积、日志把大文本全量打出来、以及 GPU 显存没有正确回收。还有一种常见问题是请求上下文里带了太多聊天历史和检索内容，导致单次请求对象特别大。处理这类问题时，我一般会先看对象生命周期、缓存策略、并发模型和监控数据，而不是先怀疑语言本身。

Q23：Python AI 项目怎么做环境和依赖管理更稳妥？

我会尽量做到解释器版本固定、依赖版本固定、环境隔离、构建过程可复现。常见做法是用虚拟环境隔离项目依赖，再配合 requirements 或 pyproject 把版本锁住，生产环境通常会放进容器里统一交付。AI 项目尤其要注意 CUDA、PyTorch、驱动版本之间的兼容性，因为这类问题一旦漂移，线上和本地很容易不一致。

Q24：Python AI 服务上线后，最应该监控哪些指标？

除了常规的 QPS、错误率、P95/P99 时延，我会重点看模型调用成功率、首 token 时延、总 token 数、单请求成本、检索命中率、工具调用成功率、队列堆积、缓存命中率和上下文长度分布。如果是智能体，还要看平均步数、异常终止率和人工接管率。因为 AI 应用的问题很多不体现在单一报错上，而是体现在质量、成本和时延一起漂移。

三、Java 与企业级 AI 应用集成

Q25：Java 在 AI 应用开发里的定位是什么？

我的理解是，Python 更偏模型侧和快速实验，Java 更偏企业级应用侧和系统集成。很多公司原来的核心业务系统、权限体系、消息系统、网关、交易链路本来就是 Java 生态，所以真正把 AI 落到生产里时，Java 往往承担 API 网关、业务编排、权限控制、审计日志、流程接入和高并发服务治理这些职责。

Q26：Spring AI 和 LangChain4j 这类框架解决了什么问题？

它们本质上是在 Java 生态里把“接模型、做提示词、接工具、接知识库、做结构化输出”这些通用能力封装起来，减少重复造轮子。Spring AI 更强调和 Spring 体系的整合，LangChain4j 更强调 Java 版本的 LLM 编排能力。面试里可以直接说，这类框架的价值不是替你设计系统，而是把模型接入层标准化。

Q27：Java 服务调用大模型接口时，最需要注意什么？

我最关注四件事：第一，连接池、超时和重试策略要合理；第二，流式输出要能稳定解析，不能把半包和断流当成正常结果；第三，要做好幂等和审计，尤其是会触发业务动作时；第四，要把模型错误和业务错误区分开。因为在企业系统里，调用模型不是“能返回文本就行”，而是要放进一个可治理、可观测、可回溯的服务链路里。

Q28：Java 里为什么很多 AI 服务会用异步或响应式方案？

因为很多 AI 场景是高延迟 I/O，不是纯 CPU 计算，比如等模型响应、等检索结果、等外部工具返回。如果全用同步阻塞，请求线程会被长时间占住，高并发下资源浪费比较明显。用 CompletableFuture、WebClient 或 Reactor 这类方案，可以把等待外部依赖的时间更高效地管理起来，尤其适合流式输出和多路并发聚合。

Q29：AI 应用里的 Java 线程池应该怎么配？

一般不能照搬普通 CRUD 服务的线程池配置，因为 AI 请求通常更慢、外部依赖更多、结果体积更大。我会先区分 CPU 任务和 I/O 任务，再按调用链路拆线程池，比如检索池、工具池、回调池分开，避免一个慢依赖把整个系统拖死。同时一定要有队列上限、拒绝策略和监控，不然高峰期很容易把内存和下游一起打爆。

Q30：Java 做 RAG 系统时，通常怎么接向量数据库？

常见做法是把文档切分、向量化、入库这部分做成离线或准实时索引流程，在线查询时由 Java 服务接收问题，做改写、检索、重排和上下文拼装，然后再把结果送给模型。向量数据库本身只是检索层的一部分，真正决定效果的其实是分块、Embedding、过滤条件、重排和上下文组装策略。

Q31：如果团队同时有 Java 和 Python，AI 项目怎么分工更合理？

我一般会建议让 Python 负责模型实验、数据清洗、评测脚本、训练微调、原型验证；让 Java 负责企业服务接入、业务编排、权限审计、稳定性治理和正式 API。这样分工的好处是两边都发挥各自优势，既能快速试错，也能稳妥上线。真正成熟的团队通常不是二选一，而是双语言协同。

Q32：Java AI 服务做流式输出时，SSE 和 WebSocket 怎么选？

如果主要是服务端单向往前端持续推送模型生成内容，SSE 通常更轻量，开发和代理层支持也更简单；如果需要真正双向实时通信，比如边说边打断、前端持续上报状态、多人协作会话，WebSocket 更合适。大多数“聊天式 AI 输出”场景其实用 SSE 就够了。

Q33：当外部模型服务不稳定时，Java 侧常见的治理手段有哪些？

常见的手段包括超时控制、限流、熔断、隔离、重试、降级和缓存。比如用 Resilience4j 这类组件做熔断和限流，把不同模型供应商隔离到不同资源池里，高风险接口启用备用模型。面试里我会强调一点，AI 服务的稳定性不只是模型本身的稳定，而是你对外部不确定性的治理能力。

Q34：Java 做 AI 应用的优势和短板分别是什么？

优势是工程体系成熟，适合复杂业务接入、权限控制、事务边界、监控治理和大型团队协作；短板是新模型、新框架、新论文能力更新通常慢于 Python 生态，很多首发工具也优先支持 Python。所以我会把 Java 看成生产集成主场，把 Python 看成模型创新主场。

四、Prompt 工程与上下文管理

Q35：什么是 Prompt Engineering？它解决的核心问题是什么？

Prompt Engineering 本质上是在不改模型参数的前提下，通过更好的任务描述、上下文组织、示例提供和输出约束，让模型更稳定地完成任务。它解决的不是“让模型突然变聪明”，而是让模型在当前能力范围内更少跑偏、更贴近业务目标。

Q36：System Prompt、User Prompt、Tool Prompt 的职责分别是什么？

System Prompt 主要定义模型的长期规则、角色边界和行为约束，相当于高优先级策略。User Prompt 是用户这一轮真正提出的任务。Tool Prompt 或工具描述则是告诉模型某个工具能干什么、输入输出是什么、什么时候该调用。简单说，System 负责原则，User 负责目标，Tool 负责能力边界。

Q37：怎么设计一个更稳定的提示词？

我一般会让提示词至少包含这几个部分：任务目标、输入背景、输出格式、边界条件、失败时怎么处理、以及必要的示例。能结构化就尽量结构化，能把模糊词换成可执行规则就尽量换掉。提示词稳定性的关键不是写得“华丽”，而是让模型少猜、多按规则执行。

Q38：Few-shot 示例为什么有效？

因为示例相当于给模型做了一个就地校准，让它更明确“这个任务到底要按什么风格和标准回答”。尤其是在标签不明显、格式要求强、边界容易歧义的任务里，Few-shot 往往比单纯增加一句解释更有效。它本质上是在上下文里给模型一个可模仿的目标分布。

Q39：复杂任务里，为什么常说要让模型先分析再作答？

因为很多复杂任务不是知识不会，而是一步出结果时容易遗漏约束。先分析再作答，相当于把问题拆成中间步骤，让模型在过程里对齐目标，准确率通常会更高。不过在线业务里我不一定要求它把完整思考过程都展示给用户，很多时候只需要系统内部用这个策略提高质量，最终对外仍然输出简洁答案。

Q40：什么是提示注入（Prompt Injection）？怎么防？

提示注入就是用户输入或外部文档里夹带了“忽略之前规则”“泄露系统提示词”“去执行危险操作”这类恶意指令，试图覆盖系统原本的控制逻辑。防御方式包括分层隔离系统规则、对外部文档做可信度判断、对高风险动作做白名单和人工确认、不要让模型直接拥有无限工具权限。我的理解是，提示注入本质上是 LLM 时代的输入安全问题，不能只靠一句“不要被攻击”来解决。

Q41：怎样让模型稳定输出结构化 JSON？

第一步是把字段、类型、是否必填定义清楚；第二步是尽量使用 Schema 约束或结构化输出接口，而不是只在自然语言里口头要求；第三步是在落库前做严格校验，发现不合法就重试或兜底。经验上看，越是需要进入下游系统的数据，越不能只相信模型“应该会按格式输出”。

Q42：长对话场景下，怎么做上下文管理？

我一般不会把全部历史无限累积，而是做分层处理：最近几轮保留原文，中期历史做摘要，长期事实抽成记忆，再结合业务状态做选择性拼接。因为上下文越长不一定越好，反而可能把噪声、旧约束和成本一起带上去。上下文管理的核心是保留“对当前任务真正有用的信息”。

Q43：提示词模板为什么要版本化管理？

因为提示词一旦进入生产环境，本质上就是业务逻辑的一部分。没有版本管理的话，效果变好了还是变差了、线上问题是谁引入的、A/B 测试用的是哪版模板，都会说不清。成熟一点的团队通常会把提示词当配置甚至当代码管理，做到可回滚、可评测、可审计。

Q44：为什么提示词工程不能替代系统工程？

因为很多问题不是提示词能解决的，比如知识过期、权限控制、工具安全、结果校验、成本控制、可观测性、异常恢复。提示词只能优化模型在已有信息和能力下的表现，但不能凭空补齐外部知识和工程约束。真正稳定的 AI 应用，一定是提示词、检索、工具、规则和监控一起工作的结果。

五、RAG 与知识增强

Q45：什么是 RAG？它适合解决什么问题？

RAG 就是 Retrieval-Augmented Generation，先检索外部知识，再让模型基于检索结果生成答案。它特别适合企业知识库问答、文档助手、客服知识支持这类知识更新快、需要引用来源、又不适合频繁微调模型的场景。

Q46：一个完整的 RAG 流程一般包含哪些步骤？

通常是文档采集、清洗、切分、向量化、建索引，然后在线阶段做查询理解、检索、重排、上下文拼接、模型生成和结果引用。很多人以为 RAG 只是“向量库加大模型”，其实真正影响效果的是整个链路的每一步质量。

Q47：文档切块为什么重要？切太大或太小分别有什么问题？

切块太大，会把很多无关信息一起送进上下文，噪声高、成本高，还会影响召回精度；切块太小，语义上下文可能断裂，模型拿到的片段不完整，回答也容易失真。所以切块本质上是在做“召回精度、语义完整性、上下文成本”三者之间的平衡，通常还要结合标题层级、段落边界和业务场景去设计。

Q48：Embedding 模型怎么选？

我会看几个维度：第一，是否支持目标语言和领域术语；第二，离线评测效果，比如召回率和命中率；第三，向量维度、推理成本和时延；第四，是否方便私有化部署。面试里可以直接说，没有“通用最优”的 embedding，只有适不适合当前数据和场景。

Q49：向量数据库在 RAG 里扮演什么角色？

它主要承担相似度检索和索引管理，把文本向量存起来，并在查询时快速找出最接近的候选文档。它解决的是“如何高效召回相关片段”，但它本身并不理解业务，也不负责最终答案质量，所以不能把效果问题全甩给向量库。

Q50：TopK 和相似度阈值应该怎么调？

TopK 决定一次拿回多少候选，阈值决定相似度低到什么程度就不要了。TopK 太小可能漏召回，太大又会引入噪声；阈值太高容易过严，太低又容易把不相关内容混进来。一般需要结合离线数据集做评测，再看线上反馈做动态调整，不能靠拍脑袋定死。

Q51：为什么很多 RAG 系统会加 Rerank？

因为第一阶段向量召回更像“粗筛”，速度快但不够精。Rerank 会把问题和候选文档更细致地放在一起比较，重新排序，把真正最相关的内容顶上来。尤其在文档很多、表述相似、领域术语密集的时候，Rerank 往往能明显提升最终答案质量。

Q52：什么是混合检索？什么时候比纯向量检索更好？

混合检索通常是把关键词检索和向量检索结合起来。因为纯向量检索更擅长语义相似，但对精确名词、编号、报错码、文件名这类词可能不够敏感；关键词检索在这些场景下反而更稳。所以如果业务里有很多精确术语、产品型号、接口名、法规编号，混合检索通常比纯向量检索更靠谱。

Q53：什么是 Query Rewrite 或 Multi-Query？

就是在正式检索前，先把用户原问题改写得更适合检索，或者扩展成多个表达方式再一起检索。因为用户提问有时很口语、很模糊，而知识库里的表达可能很正式。做 Query Rewrite 的目的不是改变用户意图，而是缩短“用户语言”和“文档语言”之间的距离。

Q54：Metadata Filter 在 RAG 里有什么作用？

它的作用是先按业务条件缩小检索范围，比如只搜某个租户、某个部门、某个时间段、某个文档类型。这样做一方面能提高检索精度，另一方面也能帮助做权限控制。很多企业级 RAG 项目里，过滤条件和权限体系往往比向量相似度本身还重要。

Q55：为什么 RAG 结果最好带引用来源？

因为带来源可以提升可信度，也方便用户追溯原文、人工核对和事后审计。尤其是企业知识问答、法务、医疗、金融这类场景，没有引用就很难判断模型到底是在复述文档还是在自由发挥。引用来源本质上是在给答案增加可验证性。

Q56：RAG 项目里最常见的效果问题有哪些？

常见问题包括切块不合理、文档清洗质量差、Embedding 不匹配、检索噪声高、重排缺失、上下文拼接过长、提示词没有明确要求基于资料回答、以及知识库更新不及时。很多团队刚开始做 RAG 时会误以为“模型回答不好”，但真正问题常常出在检索链路前面。

Q57：RAG 应该怎么评测？

我会把评测拆成两层。第一层是检索评测，比如命中率、召回率、MRR、NDCG，看有没有把相关文档找回来；第二层是生成评测，比如答案准确性、忠实性、完整性、引用是否对应原文。因为如果只看最终答案好不好，很难定位问题到底出在召回、重排还是生成。

Q58：知识库内容经常更新时，RAG 系统怎么保证新鲜度？

通常需要建立增量更新机制，而不是每次全量重建。比如文档变更后触发重新切分、重新向量化、局部更新索引；对于高频变动的数据，甚至会把向量检索和数据库实时查询结合起来。我的理解是，RAG 的核心优势之一就是知识更新不需要重新训练，所以更新链路一定要设计好。

六、智能体基础与架构设计

Q59：什么是智能体（Agent）？它和普通问答机器人有什么区别？

智能体可以理解为“带有目标、状态、工具和行动能力的大模型应用”。普通问答机器人通常是一问一答，最多做一点上下文拼接；而智能体除了回答，还会规划步骤、调用工具、读写记忆、根据环境反馈调整行为，甚至可以持续执行一个任务直到完成。

Q60：Agent 和 Workflow 的区别是什么？

Workflow 更像预先设计好的固定流程，步骤、分支和顺序相对确定，可控性强、可预测性高。Agent 更强调动态决策，它会根据中间结果决定下一步做什么，灵活性更强，但不确定性也更高。我的经验是，规则稳定的场景优先 Workflow，开放任务或复杂决策场景才考虑 Agent。

Q61：一个典型 Agent 一般由哪些组件组成？

通常至少有这几部分：目标与指令、模型本体、会话状态、记忆模块、工具集、规划策略、执行循环和结果校验。再往生产级做，还会补权限系统、审计日志、成本控制、失败恢复和人工接管机制。也就是说，Agent 不是一个单独模型，而是一套围绕模型构建的行动系统。

Q62：Agent 里的 Planning 常见有哪些模式？

比较常见的是一步一判断的短规划模式、先列计划再执行的长规划模式，以及计划执行过程中动态修正的混合模式。短规划灵活，但容易反复试探；长规划可解释性更强，但一旦前提错了，后面可能全偏。真实项目里常见做法是先给一个粗计划，再允许根据执行反馈局部调整。

Q63：什么是 ReAct 思路？为什么它很常见？

ReAct 可以概括成“先推理，再行动，再观察结果，再继续推理”。它常见是因为这个模式很符合智能体和外部世界交互的过程：模型先判断下一步需要什么，再去调用工具，拿到结果后继续往前走。它的优势是透明、可迭代，但缺点是链路变长以后，时延和成本也会增加。

Q64：Agent 为什么要用工具，而不是所有事情都让模型直接回答？

因为模型擅长生成和归纳，不擅长直接获得实时事实、精确计算、数据库写操作和外部系统动作。如果把所有事情都交给模型“猜”，准确率和安全性都会出问题。工具的意义就是把那些需要真实世界交互的能力外接出来，让模型做决策，让工具做执行。

Q65：函数调用和 MCP 这类协议在 Agent 里是什么关系？

函数调用更像模型侧的能力，意思是模型能够按指定 Schema 产出结构化调用意图。MCP 更像系统侧的标准协议，目的是把工具、资源和提示能力用统一方式暴露给模型应用。简单说，函数调用解决“模型怎么表达要调什么”，MCP 解决“外部能力怎么被标准化接入”。

Q66：Agent 的记忆一般分哪几类？

常见可以分成短期记忆和长期记忆。短期记忆主要是当前任务相关的上下文、最近几轮对话和中间结果；长期记忆主要是用户偏好、稳定事实、项目约定和长期经验。有的系统还会再区分工作记忆、情节记忆和语义记忆，但本质上都是在回答一个问题：什么信息该临时保留，什么信息值得跨会话持续保留。

Q67：单智能体和多智能体分别适合什么场景？

单智能体适合大多数中小型任务，架构简单，调试和治理成本低。多智能体更适合任务天然可分工，比如一个负责检索，一个负责编码，一个负责审查，一个负责汇总。它的优势是专业化和并行化，但代价是通信复杂度、上下文同步成本和失控风险都会上升。

Q68：Agent Loop 最大的风险是什么？

最大的风险不是“回答错一句话”，而是它可能在错误前提下持续做错事，比如反复调用无效工具、不断扩大上下文、重复执行同一动作、或者在高权限环境里做出不该做的操作。所以 Agent Loop 一定要有步数上限、工具权限边界、状态检查和中止条件。

Q69：怎么防止智能体陷入无限循环？

常见做法是限制最大步数、限制同类工具连续调用次数、检测重复动作、要求每一步都说明目标是否推进、并在关键节点加入停止判定。如果连续几步都没有新信息或者结果没有改善，就应该强制结束或请求人工介入。我的理解是，智能体不是越“自主”越好，而是越“可控”越好。

Q70：怎么判断一个 Agent 的任务拆解能力好不好？

我会看三个维度。第一，拆出来的步骤是不是覆盖了任务关键路径；第二，步骤之间是否有依赖顺序和可执行性；第三，执行中遇到失败时能不能重新规划，而不是硬着头皮继续。真正好的任务拆解不是把任务拆得很碎，而是拆得刚好能执行、能验证、能回滚。

Q71：什么场景必须加 Human-in-the-Loop？

凡是涉及真实外部动作、高权限操作、资金流转、对外发送、合同和法律风险、生产数据改写的场景，我都会加人工确认。因为这类场景的错误成本太高，不能只靠模型概率判断。面试里我通常会说，人审不是为了削弱智能体，而是为了把不可逆风险拦在系统外面。

Q72：做智能体项目时，最常见的失败原因是什么？

最常见的失败原因不是模型不够强，而是目标定义不清、工具设计太粗糙、状态管理混乱、权限边界缺失、评测闭环不足。还有很多项目一开始就把智能体用在不适合开放决策的场景里，结果复杂度远大于收益。我的经验是，智能体项目成败往往取决于系统设计，而不是一句提示词。

七、工具调用与系统集成

Q73：什么是 Function Calling？它为什么重要？

Function Calling 就是让模型不要只输出自然语言，而是按预定义的参数结构告诉系统“我想调用哪个函数、传什么参数”。它的重要性在于把模型从“只能聊天”变成“可以驱动系统能力”的入口，同时也让参数校验、审计和权限控制变得更可做。

Q74：MCP 的价值主要体现在哪？

它的价值在于把原来零散、各写各的工具接入方式标准化，让模型应用可以更一致地接资源、接工具、接上下文能力。对企业来说，这样做的好处是接入成本更低、复用性更强、治理边界更清楚。简单说，MCP 不直接提升模型智商，但能提升整个工具生态的可组合性。

Q75：设计工具 Schema 时，最重要的原则是什么？

我会强调四个原则：语义清晰、参数最小化、边界明确、可校验。工具描述一定要让模型知道这个工具能干什么、不能干什么、什么时候该调用；参数不要把不必要的自由度放给模型；返回值要尽量结构化，方便后续判断。Schema 设计得越清楚，模型误用工具的概率就越低。

Q76：为什么智能体工具必须做权限分级？

因为读操作和写操作、查询操作和对外执行操作、低风险动作和高风险动作，本来就不应该由同一套权限控制。权限分级可以避免模型在一次错误判断后直接造成不可逆后果。对高风险工具，我通常会加白名单、参数校验、二次确认甚至人工审批。

Q77：智能体调用外部 API 时，为什么要考虑幂等性？

因为网络超时、重试、重复调用在分布式系统里很常见。如果一个创建订单、发消息、提交工单的接口不具备幂等性，智能体一旦重试就可能产生重复操作。面试里可以直接说，模型调用工具和普通服务调用一样，最终还是要回到分布式系统的基本原则上来。

Q78：让智能体读写文件、改代码时，安全上要注意什么？

核心是最小权限原则和可审计原则。首先只开放必要目录和必要动作，不要默认全盘可写；其次要区分扫描、读取、修改、删除这几类风险；再就是高风险动作必须先列出影响范围并确认。因为一旦把文件系统作为工具暴露给智能体，它面对的就不是“文本生成”，而是真实的系统副作用。

Q79：浏览器自动化在智能体里适合哪些场景？风险是什么？

它适合处理必须基于页面交互的任务，比如后台录入、网页测试、抓取动态页面信息、流程演示。风险主要在于页面结构容易变、登录态容易失效、动作不可预测、以及可能误触真实业务按钮。我的看法是，浏览器自动化应该优先用于低风险、可回放、可验证的任务，而不是直接拿去做高风险生产操作。

Q80：为什么很多智能体任务要改成异步任务或后台任务？

因为有些任务天然就不是秒级完成的，比如全库检索、批量生成文档、多步工具链执行、长时间网页操作。如果硬塞进一次同步请求里，用户体验差，服务资源也会被长时间占住。把它做成异步任务后，可以让前端先拿到任务 ID，再轮询状态或接收回调，整体系统会更稳。

Q81：AI 应用为什么特别需要缓存和限流？

因为模型调用贵、慢，而且下游依赖通常也多。缓存可以减少重复请求成本，比如重复问答、重复检索、重复 embedding；限流则是防止用户突发流量把模型配额、线程池和下游工具一起冲垮。对 AI 系统来说，缓存和限流不只是性能优化，更是成本治理和稳定性治理。

Q82：AI 应用接企业内部系统时，最大的挑战通常是什么？

通常不是“怎么调 API”，而是权限、数据口径、幂等、审计、流程边界和责任归属。企业系统往往有严格的主数据定义和操作约束，如果智能体绕过这些规则，效果再聪明也落不了地。所以我会把企业集成理解为“把 AI 放进原有治理体系”，而不是“让 AI 凌驾于原有系统之上”。

八、多智能体、记忆与工作流编排

Q83：短期记忆和长期记忆的本质区别是什么？

短期记忆服务的是当前任务，强调即时性和相关性，比如这一轮会话、当前计划、中间观察结果。长期记忆服务的是跨任务复用，强调稳定性和持续价值，比如用户偏好、组织术语、项目约定。它们的区别不只是保存时间长短，更在于“是否值得跨会话保留”。

Q84：为什么记忆不能什么都存？

因为什么都存会把噪声、错误和隐私风险一起放大。记忆系统最怕两件事，一是把临时信息当长期事实，二是把未经验证的信息反复带入后续任务。好的记忆写入策略应该是少而准，只保留跨会话真正有价值、且可信度足够高的信息。

Q85：什么时候值得用多智能体协作？

当任务天然存在角色分工、不同阶段需要不同能力、或者并行执行能显著节省时间时，多智能体才有意义。比如研究、编码、审查、汇报四类任务明显不同，就可以拆成不同代理协作。反过来说，如果一个代理已经能清楚完成，就没必要为了“看起来高级”硬上多智能体。

Q86：多智能体之间应该怎么通信才更稳？

我更倾向于让它们传递结构化消息，而不是随意长篇对话。比如明确任务目标、输入、输出格式、状态、是否完成、需要谁接力。因为代理一多，上下文漂移和责任不清会非常快地放大，所以通信协议越明确，系统越稳定。

Q87：工作流引擎和 Agent 框架分别适合干什么？

工作流引擎适合做固定流程编排、状态流转、重试补偿、定时任务和可视化流程管理；Agent 框架更适合处理开放决策、工具选择和动态规划。我的经验是，生产环境里经常是工作流做骨架，Agent 做局部高自由度决策，两者并不是互斥关系。

Q88：什么是事件驱动的 AI 工作流？

就是把 AI 任务的启动和推进建立在事件上，比如用户上传文档、工单状态变化、数据库有新增记录、某个任务执行完成后触发下一步。这样做的好处是系统更解耦，也更适合长链路、多步骤和异步场景。很多真正上线的 AI 业务，并不是一轮对话解决全部，而是由多个事件逐步推进。

Q89：什么场景其实不适合用智能体？

如果流程固定、规则明确、结果格式稳定、出错成本又高，那我通常更建议用普通程序或工作流，而不是智能体。比如固定报表生成、标准字段映射、明确规则审批，这类事情不需要让模型自由发挥。判断是否适合用智能体，一个很实用的标准就是：这件事到底需要动态决策，还是只是流程自动化。

Q90：做 Agent 产品时，除了准确率，还应该关注哪些业务指标？

我会关注任务完成率、人工接管率、平均步数、单任务成本、总耗时、工具成功率、重复调用率、用户复用率和最终业务转化。因为智能体不是纯模型评测题，真正上线后更重要的是它到底有没有帮业务完成任务、有没有把人工成本降下来。

九、评测、安全、部署与面试实战

Q91：AI 应用应该怎么做评测？

我一般会把评测分成离线集评测和线上真实反馈两部分。离线评测负责可重复、可对比，适合看不同提示词、不同检索策略、不同模型的差异；线上评测负责看真实用户是否买账，包括点击、追问、人工接管和满意度。只有离线没有线上，容易脱离真实业务；只有线上没有离线，又很难快速定位问题。

Q92：离线评测和在线评测的区别是什么？

离线评测用固定数据集，优点是稳定、可复现、方便做 A/B 对比；缺点是可能覆盖不到真实世界的复杂输入。在线评测直接看线上效果，优点是最贴近真实业务，缺点是成本更高、变量更多。我的做法一般是离线先筛方案，线上再灰度验证。

Q93：AI 应用的时延一般从哪些地方优化？

通常从四个方向优化：第一，减少无效上下文和不必要步骤；第二，优化检索和工具链路，比如并发化、缓存化；第三，模型侧选更合适的尺寸、量化和流式输出；第四，网络侧减少跨区域和重复连接。时延优化不能只盯模型本身，很多时候真正慢的是检索、工具和串行编排。

Q94：AI 应用怎么做成本优化？

核心思路是少调、少传、少生成、调对模型。比如能缓存就缓存，能先做轻量分类就不要一上来用大模型，能缩短上下文就不要把整份文档塞进去，能用小模型做初筛就不要所有环节都上最贵的模型。成本优化本质上不是“压榨模型”，而是让系统分层处理不同复杂度的问题。

Q95：AI 应用里的安全问题一般分哪几类？

常见可以分成输入安全、输出安全、工具安全、数据安全和模型安全。输入安全包括提示注入和恶意内容；输出安全包括违规回答、事实错误、泄露敏感信息；工具安全涉及越权调用和危险操作；数据安全涉及隐私、权限和审计；模型安全还包括供应链、模型投毒和越狱风险。面试里如果能从“系统安全”而不是只从“内容安全”去答，会更完整。

Q96：企业做 AI 应用时，数据隐私和合规通常要关注什么？

首先要明确哪些数据能进模型、哪些必须脱敏、哪些不能离开内网；其次要有日志审计和权限控制，保证谁看了什么、谁调用了什么都能追踪；再就是供应商合规、数据保留期限和跨境传输要求。简单说，企业不是不能做 AI，而是必须在数据治理框架内做 AI。

Q97：开源模型和闭源模型应该怎么选？

如果业务更看重效果上线快、通用能力强、无需自己维护底层推理，闭源模型通常更省事；如果更看重数据可控、成本可控、可私有化和深度定制，开源模型更有优势。面试里我一般不会说谁绝对更好，而是会从能力、成本、时延、合规和可控性几个维度做取舍。

Q98：一个生产级 AI 应用的部署架构应该考虑哪些点？

我会重点考虑模型接入方式、服务拆分、缓存层、队列层、检索层、存储层、鉴权层、监控告警和灰度发布。除此之外，还要考虑多模型切换、故障降级、限流熔断、日志审计和弹性扩缩容。生产级架构的关键不是“能跑起来”，而是高峰能扛住、出错能止损、上线能回滚。

Q99：AI 应用线上出事故时，你一般怎么排查？

我会先判断是全局问题还是局部问题，再按链路拆分：请求入口、权限校验、检索、工具、模型调用、后处理、前端展示。然后结合日志、trace、指标看是质量下降、时延升高还是错误率升高。AI 事故很容易被误判成“模型不行”，但很多时候其实是索引没更新、外部工具超时、提示词改坏了，或者上下文拼装出了问题。

Q100：面试里如果让你介绍一个“AI 应用或智能体项目”，你会怎么讲？

我会按“业务目标、系统架构、关键难点、核心取舍、效果数据、线上问题与优化”这六部分来讲。先说这个项目解决了什么业务问题，再说整体链路是模型、检索、工具和业务系统怎么协同的，然后重点讲我真正负责的难点，比如提示词、RAG、工具治理、时延优化或安全控制。最后补上结果，比如完成率、时延、成本、人工节省比例，以及我做过哪些迭代优化。这样讲能让面试官快速判断我不是只会调 API，而是真的做过完整系统。
